services, you can build more sophisticated and resilient Lambda-based solutions that can handle complex business processes and react to a wide range of system and application events.

For more information, see Managing Lambda workflows and events.

Developing Lambda functions locally with VS Code

You can move your Lambda functions from the Lambda console to Visual Studio Code, which provides a full development environment and allows you to use other local development options like AWS SAM and AWS CDK.

Key benefits of local development

While the Lambda console provides a quick way to edit and test functions, local development offers more advanced capabilities:

- **Advanced IDE features**: Debugging, code completion, and refactoring tools
- **Offline development**: Work and test changes locally before cloud deployment
- **Infrastructure as code integration**: Seamless use with AWS SAM, AWS CDK, and Infrastructure Composer
- **Dependency management**: Full control over function dependencies

Prerequisites

Before developing Lambda functions locally in VS Code, you must have:

- **VS Code**: For installation instructions, see Download VS Code.
- **AWS Toolkit for Visual Studio Code**: For installation instructions, see Setting up the AWS Toolkit for Visual Studio Code. For an overview, see AWS Toolkit for Visual Studio Code.
- **AWS credentials**: For information about configuring credentials, see Setting up your AWS credentials.
- **AWS SAM CLI**: For installation instructions, see Installing the AWS SAM CLI.
- **Docker installed (optional, but required for local testing)**: For installation instructions, see Get Docker.

> **Note**
>
> If you already have an AWS account and profile configured locally, ensure that the AdministratorAccess managed policy is added to your configured AWS profile.

Authentication and access control

To develop Lambda functions locally, you need AWS credentials to securely access and manage AWS resources on your behalf, just like they would in the cloud. The AWS Toolkit for VS Code supports the following authentication methods:

The AWS Toolkit for VS Code supports the following authentication methods:

- IAM user long-term credentials
- Temporary credentials from assumed roles
- Identity federation
- AWS account root user credentials (not recommended)

This section guides you through obtaining and configuring these credentials using IAM user long-term credentials.

Get IAM Credentials

If you already have an IAM user with access keys, have both the access key ID and secret access key ready for the next section. If you don't have these keys, follow these steps to create them:

> **Note**
>
> You must use both the access key ID and secret access key together to authenticate your requests.

To create an IAM user and access keys:

1. Open the IAM console at https://console.aws.amazon.com/iam/
2. In the navigation pane, choose **Users**.

3. Choose **Create user**.
4. For **User name**, enter a name and choose **Next**.
5. Under **Set permissions**, choose **Attach policies directly**.
6. Select **AdministratorAccess** and choose **Next**.
7. Choose **Create user**.
8. In the success banner, choose **View user**.
9. Choose **Create access key**.
10. For **Use case**, select **Local code**.
11. Select the confirmation check box and choose **Next**.
12. (Optional) Enter a description tag value.
13. Choose **Create access key**.
14. Copy your access key and secret access key immediately. **You won't be able to access the secret access key again after you leave this page.**

> **Important**
>
> Never share your secret key or commit it to source control. Store these keys securely and delete them when no longer needed.

> **Note**
>
> For more information, see Create an IAM user in your AWS account and Manage access keys for IAM users in the *IAM User Guide*.

Configure AWS credentials using the AWS Toolkit

The following table summarizes the credential setup process you will complete in the following procedure.

What to Do	Why?
Open Sign In panel	Start authentication

What to Do	Why?
Use Command Palette, search for AWS Add a New Connection	Access the sign-in UI
Choose IAM Credential	Use your access keys for programmatic access
Enter profile name, access key, secret key	Provide credentials for connection
See AWS Explorer update	Confirm you're connected

Complete the following steps authenticate to your AWS account:

1. Open the Sign In panel in VS Code:
   a. To start the authentication process, select the AWS icon in the left navigation pane or open the Command Palette (Cmd+Shift+P on Mac or Ctrl+Shift+P on Windows/Linux) and search for and select **AWS Add a New Connection**.
2. In the sign in panel, choose **IAM Credentials** and select **Continue**.

> **Note**
>
> To proceed, you will need to allow AWS IDE Extensions for VS Code to access your data.

3. Enter your profile name, access key ID, and secret access key, then select **Continue**.
4. Verify the connection by checking the AWS Explorer in VS Code for your AWS services and resources.

For information on setting up authentication with long-term credentials, see Using long-term credentials to authenticate AWS SDKs and tools.

For information about configuring authentication, see AWS IAM credentials in the AWS Toolkit for Visual Studio Code User Guide.

Moving from console to local development

> **Note**
>
> If you've made changes in the console, make sure you don't have any undeployed changes before transitioning to local development.

To move a Lambda function from the Lambda console to VS Code, complete the following steps:

1. Open the Lambda console.
2. Choose the name of your function.
3. Select the **Code source** tab.
4. Choose **Open in Visual Studio Code**.

   > **Note**
   >
   > The **Open in Visual Studio Code** button is only available in AWS Toolkit version **3.69.0** and later. If you have an earlier version of the AWS Toolkit installed, you may see a `Cannot open the handler` message in VS Code. To resolve this, update your AWS Toolkit to the latest version.

5. When prompted, allow your browser to open VS Code.

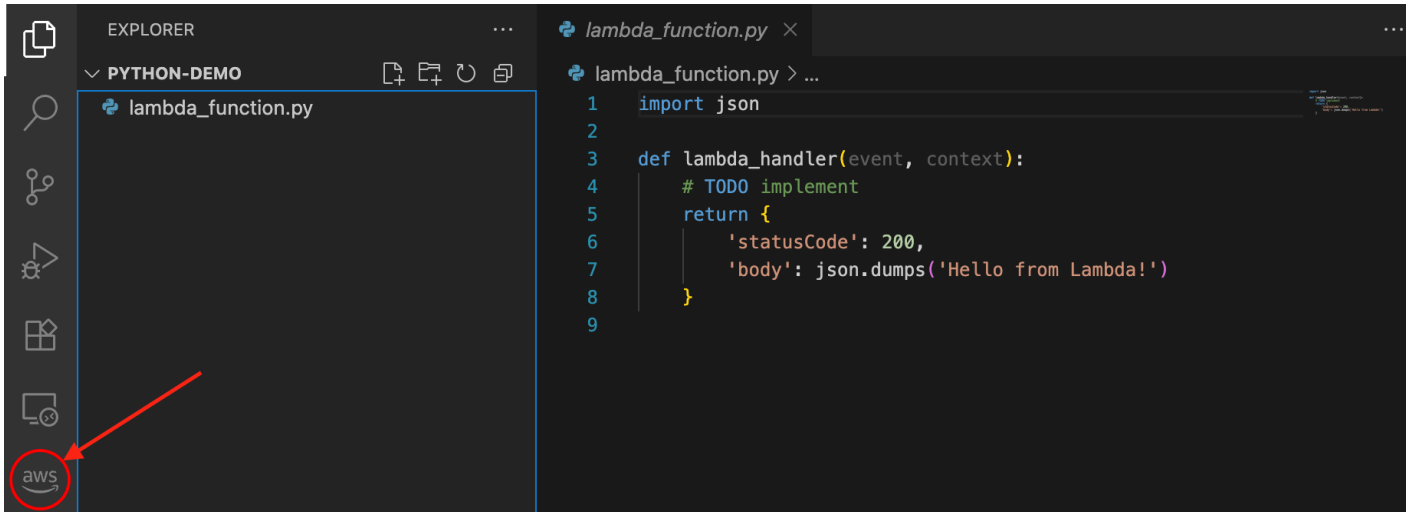
When you open your function in VS Code, Lambda creates a local project with your function code in a temporary location that's designed for quick testing and deployment. This includes the function code, dependencies, and a basic project structure that you can use for local development.

For details on using AWS in VS Code, see the *AWS Toolkit for Visual Studio Code User Guide*.

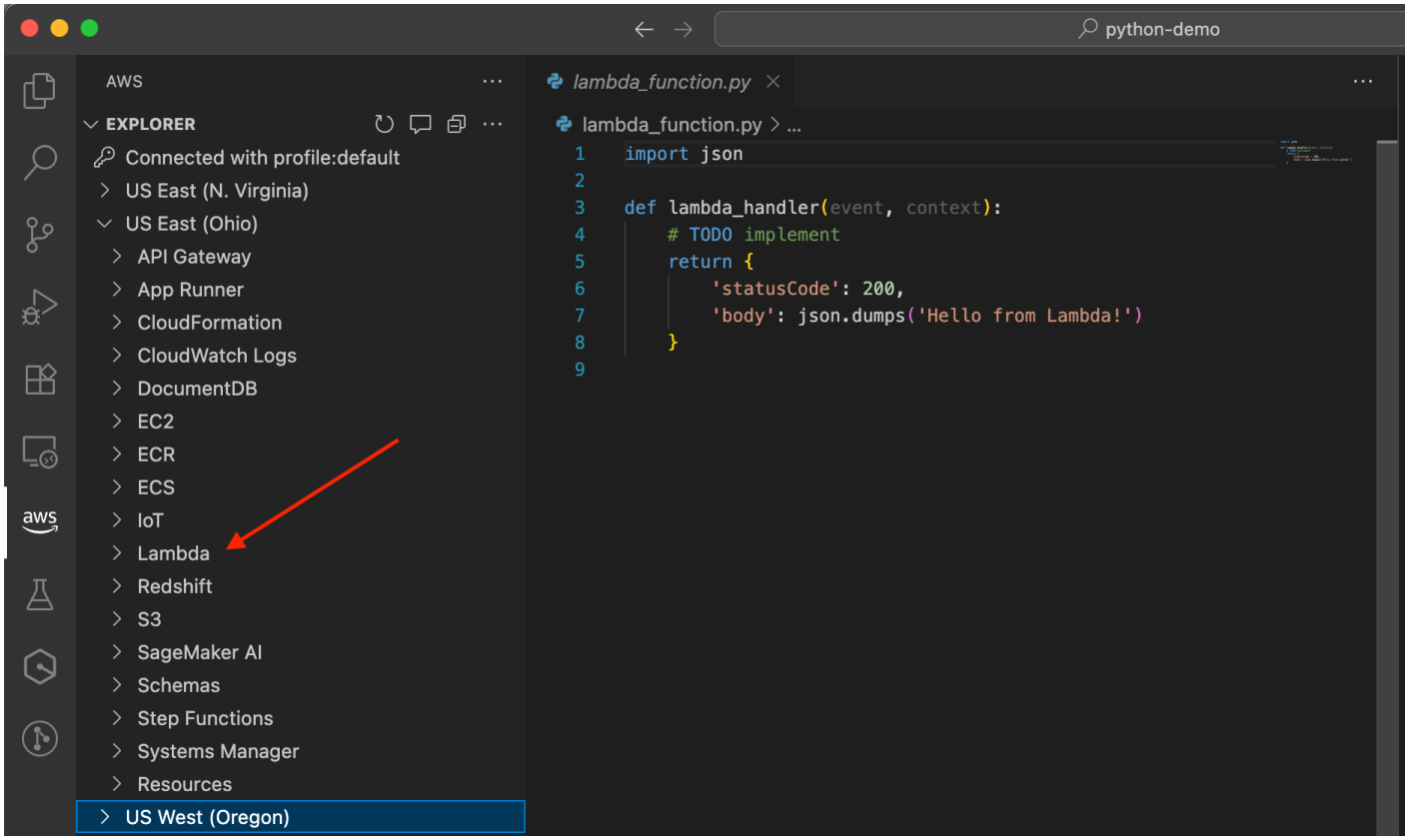
Working with functions locally

After opening your function in VS Code, follow these steps to access and manage your functions:

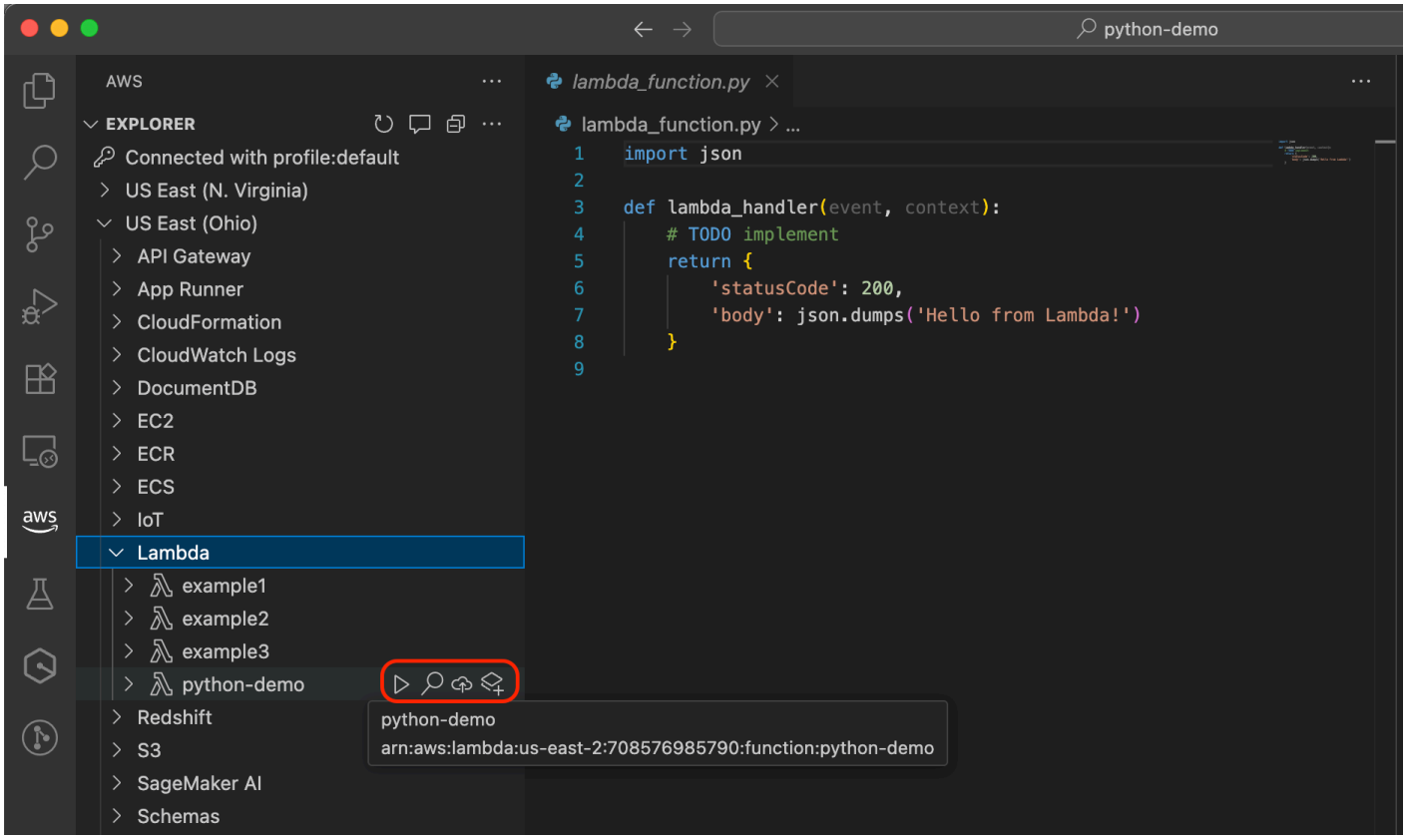
1. Select the AWS icon in the sidebar to open the AWS Explorer:



2. In the AWS Explorer, select the region with your Lambda function:



3. Under your selected region, expand the Lambda section to view and manage your functions:



With your function opened in VS Code, you can:

- Edit function code with full language support and code completion.
- Use the LocalStack integration in VS Code to test Lambda functions that make API calls to other AWS services during execution, such as reading from DynamoDB tables or writing to Amazon S3 buckets. LocalStack is a cloud service emulator that provides a complete local development environment for testing service integrations. You can also use AWS SAM CLI to test your function in a local container. If your function makes API calls to other AWS services those calls will reach real AWS resources, not emulated ones.
- Debug your function with breakpoints and variable inspection. For more information, see Running and debugging Lambda functions directly from code in the *AWS Toolkit for Visual Studio Code User Guide*.
- Deploy your updated function back to AWS using the cloud icon.
- Install and manage dependencies for your function.

For more information, see Working with AWS Lambda functions in the AWS Toolkit for Visual Studio Code User Guide.

Convert your function to an AWS SAM template and use IaC tools

In VS Code, you can convert your Lambda function to an AWS SAM template by choosing the **Convert to AWS SAM Application** icon next to your Lambda function. You will be prompted to select an AWS SAM project location. Once selected, your Lambda function will be converted to a `template.yaml` file that is saved in your new AWS SAM project.

With your function converted to an AWS SAM template, you can:

- Control the versioning of your infrastructure
- Automate deployments
- Remotely debug functions
- Add additional AWS resources to your application
- Maintain consistent environments across your development lifecycle
- Use Infrastructure Composer to visually edit your AWS SAM template

For more information on using IaC tools, refer to the following guides:

- The AWS Serverless Application Model Developer Guide
- The AWS Cloud Development Kit (AWS CDK) Developer Guide
- The Infrastructure Composer Developer Guide
- The AWS CloudFormation User Guide

These tools provide additional capabilities for defining, testing, and deploying your serverless applications.

Next steps

To learn more about working with Lambda functions in VS Code, see the following resources:

- Working with AWS Lambda functions in the AWS Toolkit for VS Code User Guide
- Working with serverless applications in the AWS Toolkit for VS Code User Guide